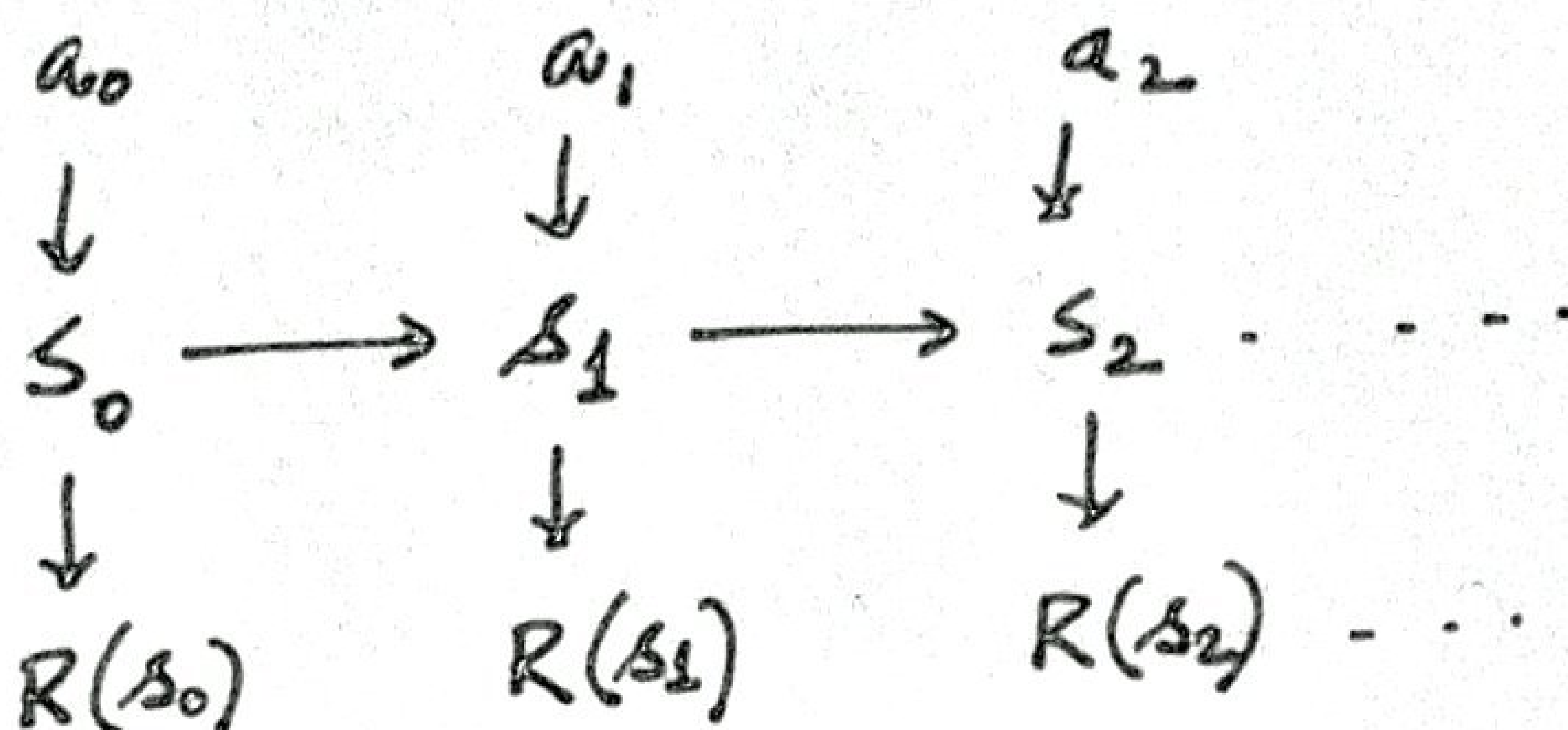


Reinforcement Learning

CS229.

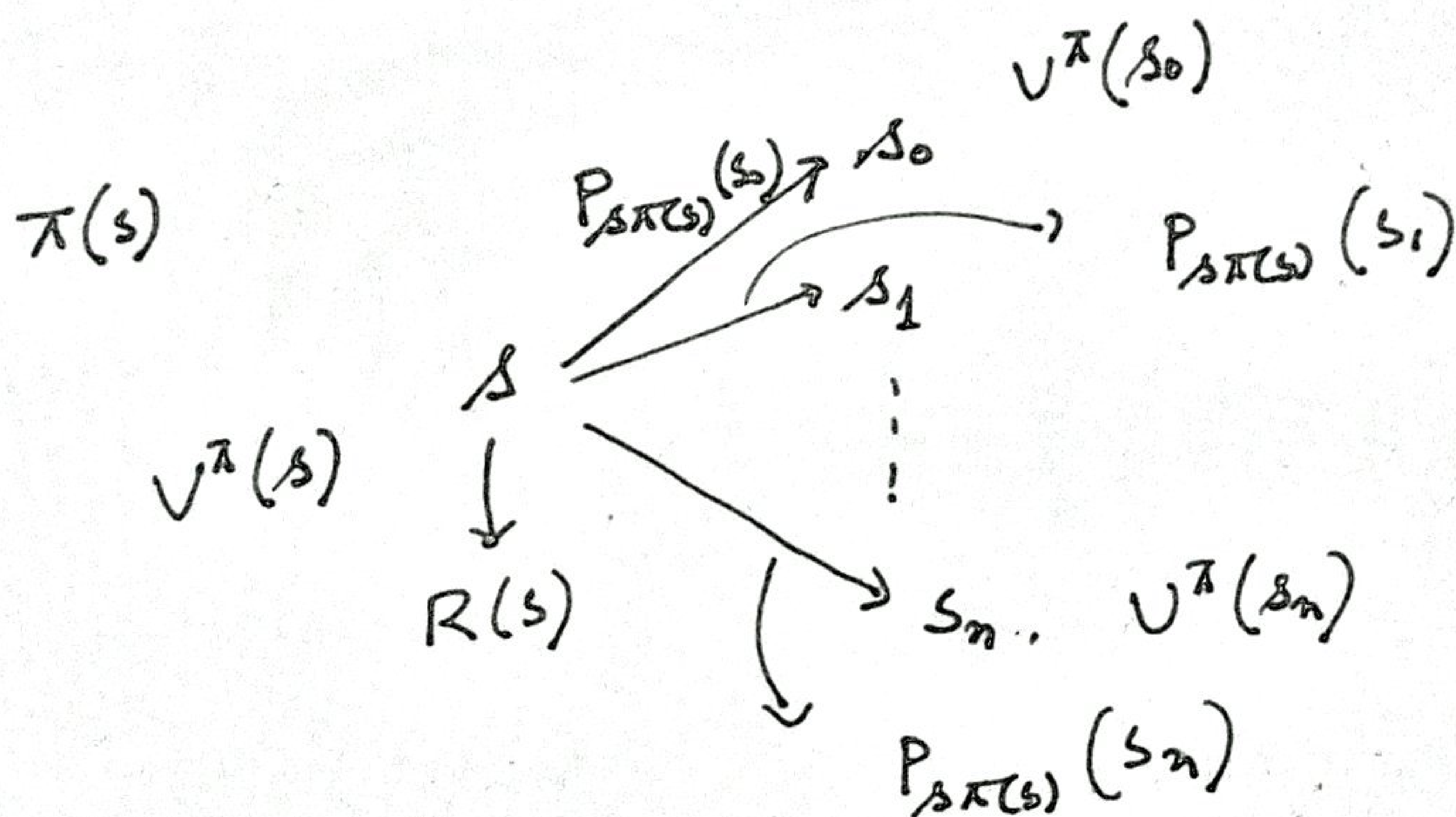


Goal of RL is to maximise:

$$E[R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \dots]$$

Let say, we are following policy π :

$$V^\pi(s) = E[R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \dots | s = s_0, \pi]$$



$$V^\pi(s) = R(s) + \gamma \left[P_{s\pi(s)}(s_0) \cdot V^\pi(s_0) + P_{s\pi(s)}(s_1) \cdot V^\pi(s_1) + \dots + P_{s\pi(s)}(s_n) \cdot V^\pi(s_n) \right]$$

$$= R(s) + \gamma \sum_{s' \in S} V^\pi(s') P_{s\pi(s)}(s')$$

Where $S = \{s_0, s_1, \dots, s_n\}$.

This equation is known as Bellman Equation.

⊙ Bellman's equation can be solved when $|S| < \infty$. $|S|$ No. of variable & $|S|$ no. of equations

In the lab sheet, there are 3 equations.

$V^\pi(s_1), V^\pi(s_2); V^\pi(s_3)$: linear equation

⊙ Obtaining optimal value function:

$$V^*(s) = \max_{\pi} [V^\pi(s) + \gamma \sum_{s'} P(s'|s, \pi) V^*(s')] \quad \forall s \in S$$

Need to find $V^*(s)$ for all $s \in S$. This is a system of linear equations. We can solve it by finding the optimal policy π^* and then computing $V^{\pi^*}(s)$.

$S \rightarrow s_1 \rightarrow s_2 \rightarrow \dots \rightarrow s_K$

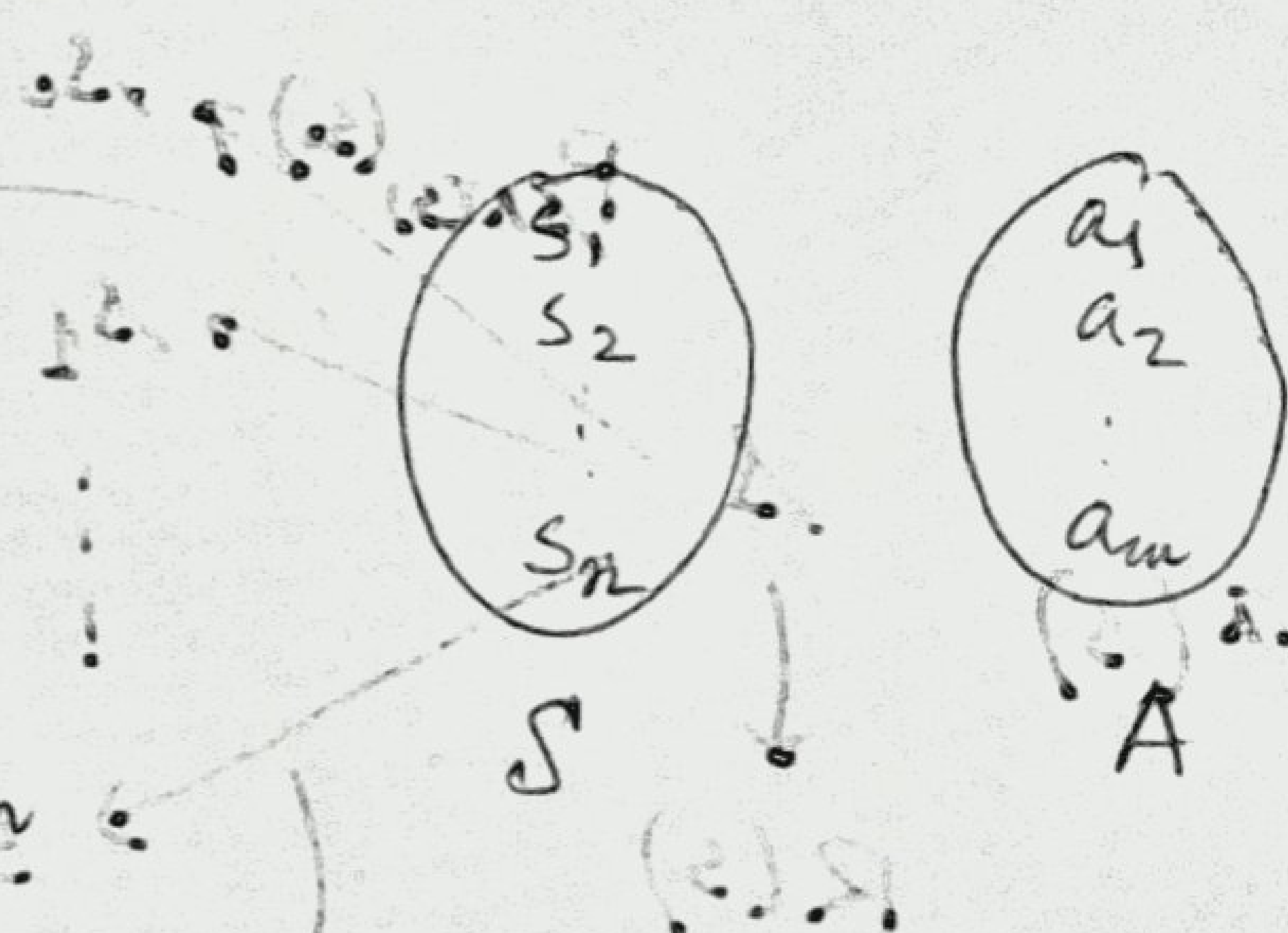
π_1
 π_2
 $\vdots$
 π_K

$V^{\pi_1}(s)$

$V^{\pi_2}(s)$

$V^{\pi_K}(s)$

$K = m^n$



$$\pi^* = \arg \max_{\pi} \{ V^{\pi_1}(s), V^{\pi_2}(s), \dots, V^{\pi_K}(s) \}$$

$m \times m \times \dots \times n$

$= m^n$

$V^{\pi_2}(s)$

$V^{\pi_K}(s)$

$$V^*(s) = \max_{\pi} V^{\pi}(s)$$

$$= \max_{\pi} \left[R(s) + \gamma \sum_{s' \in S} V^{\pi}(s') P_{s\pi(s)}(s') \right]$$

$$= R(s) + \gamma \max_{\pi} \sum_{s' \in S} V^{\pi}(s') P_{s\pi(s)}(s')$$

$$= R(s) + \gamma \left(\max_{a \in A} \sum_{s' \in S} P_{sa}(s') \cdot V^*(s') \right)$$

$$\pi^*(s) = \arg \max_{a \in A} \sum_{s' \in S} P_{sa}(s') \cdot V^*(s')$$

The optimal policy: $\pi^*(s)$ gives the action a that attains the maximum in the "max" expression.

$$V^*(s) = V^{\pi^*}(s) \geq V^{\pi}(s)$$

π^* is the optimal policy for all states.

It is not the case that if we were starting in some other state s' then there'd be some other policy that's optimal policy for s' . The same policy π^* attains the maximum for all the states.

* Policy iteration

Initialize π randomly.

for (until convergence) do

Let $V := V^\pi$

For each state s , let

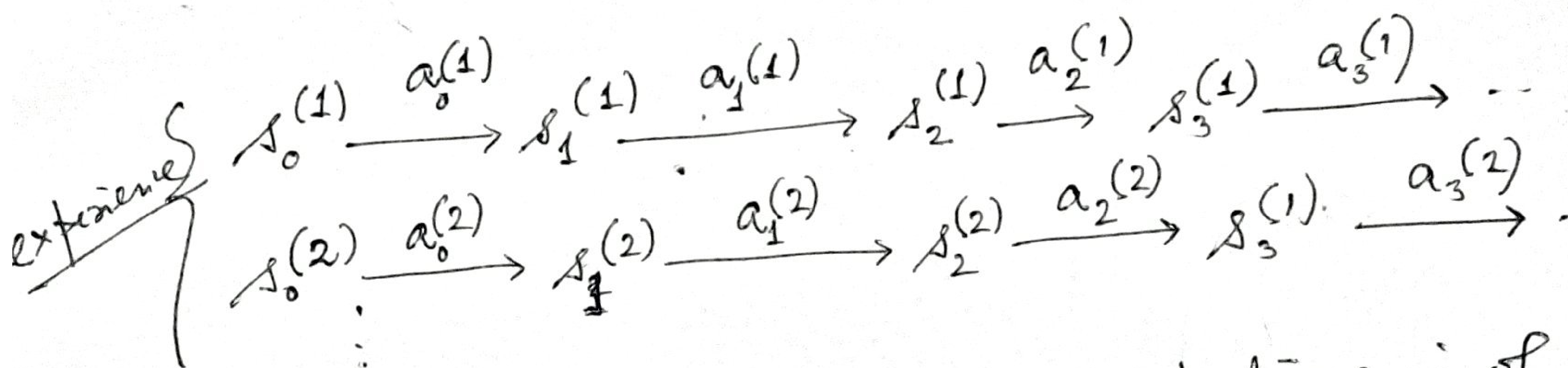
$$\pi(s) = \arg \max_{a \in A} \sum_{s'} P_{sa}(s') V(s')$$

$$V^*(s) = V^{\bar{A}^*}(s)$$

π^*

Learning a model for an MDP

In realistic problems, we are not given state transition probabilities and rewards explicitly but instead estimate them from data.



$s_i^{(j)}$ is the state we were at time i of trial j , and $a_i^{(j)}$ is the corresponding action that was taken from that state.

Given this "experience" in the MDP, consisting of a number of trials, we can estimate state transition probabilities as follows:

$$P_{sa}(s') = \frac{\text{\# times we took action } a \text{ in state } s \text{ and got to } s'}{\text{\# times we took action } a \text{ in state } s.}$$

* If the ratio is " $\frac{0}{0}$ ", then the probability measure is $\frac{1}{|S|}$.

⑧ Reward (s) =
$$\frac{\sum \text{Rewards obtained from state } s \text{ in all occurrences}}{\# \text{ occurrences of } s}$$

⑨ MDP example (Recycling Robot).

Ref. Sutton & Barto / Page No. 67

The robot has to decide (based on the charge level of the battery)

- ↗ Actively search for a CAN
- Remain stationary and wait for some ~~body~~ one to bring it a can
- ↘ Go back to home base to recharge the battery.

* The best way to find cans is to actively search for them, but this runs down the robot's battery, whereas waiting does not.

* Whenever the robot is searching, the possibility exists that its battery will become depleted. In this case, the robot must shut down and wait to be rescued.

$A(\text{high}) = \{\text{Search, wait}\}$

$A(\text{low}) = \{\text{Search, wait, recharge}\}$

$\downarrow \quad \downarrow \quad \downarrow$
 $R_s \quad R_w \quad 0$

$R_s \geq R_w$

s	s'	a	$P(s' s,a)$	$r(s,a,s')$
high	high	search	α	R_s
high	low	search	$1-\alpha$	R_s
low	low	search	β	R_s
low	high	search	$1-\beta$	-3
high	high	wait	1	R_w
high	low	wait	0	R_w
low	high	wait	0	R_w
low	low	wait	1	R_w
low	high	recharge	1	0
low	low	recharge	0	0

